Transformer-Based Unit Test Generation

Khushnood Adil Rafique, Pooja Biradar, Christoph Grimm
University of Kaiserslautern-Landau (RPTU), Chair of Cyber-Physical Systems, Kaiserslautern, Germany
[khushnood.rafique|grimm]@cs.uni-kl.de, biradar@rptu.de

***Abstract*—Automated test case generation reduces the manual effort of software testing, particularly for dynamic languages like Python. This study compares three transformer-based models—CodeT5, CodeBERT, and CodeGen. While CodeBERT and CodeGen were expected to excel due to their focus on code understanding and generation, CodeT5, a code summarization model, outperformed both, achieving higher code coverage and superior semantic and syntactic fidelity. Using a limited yet diverse dataset of Python functions with natural language descriptions and unit tests, we evaluated models with NLP metrics (BLEU, ROUGE-L) and practical measures like code coverage. CodeGen struggled with coherence under data constraints, while CodeBERT showed moderate effectiveness. Paired t-tests confirmed CodeT5's statistically significant advantage, highlighting that a well-adapted model can outperform more generalized architectures in realistic software testing scenarios.**

***Index Terms*—Natural Language Processing, Automatic Unit Test Generation, Large Language Models, Transformers, CodeT5, CodeBERT, CodeGen, Artificial Intelligence**

I. Introduction

Software testing, particularly unit testing, is a critical bottleneck in the software development lifecycle. Manual creation of unit tests is time-consuming, but AI and NLP technologies, especially transformers and LLMs, offer the potential to automate and improve test generation. Models like CodeGen [1] and CodeBERT [2] are designed for multi-language code generation and code representation, respectively, but our findings show that CodeT5 [3], a general-purpose encoder-decoder model, outperforms them in generating executable, semantically aligned test cases. CodeT5 excels across metrics, such as semantic similarity, code coverage, and statistical significance, even on a small dataset. AI-driven approaches in software testing are crucial as systems become more complex, reducing developer effort and human error. Our study shows that CodeT5's specialized pretraining for code transformation tasks, combined with its encoder-decoder architecture that leverages bidirectional context to align input functionality with test logic, enables it to outperform broader models, especially in data-constrained conditions. This research demonstrates the value of combining syntactic, semantic, and runtime evaluations for a comprehensive approach to automated testing. By challenging preconceptions and emphasizing curated datasets, this work highlights AI's transformative role in software testing, laying the groundwork for future advancements in automated testing for Python and beyond.

II. Related Work

Automated test case generation has advanced significantly, leveraging deep learning (DL) and transformer-based models to improve test quality and efficiency. Early tools like Pynguin [4] and AthenaTest [5] laid the groundwork, with Pynguin optimizing code coverage through search-based methods and AthenaTest pioneering transformer-driven unit test generation. However, AthenaTest's lack of assertion knowledge and inconsistent test signatures limited its effectiveness, with fewer than 20% of generated test cases being correct. Recent models have addressed these shortcomings. A3Test [6] integrated assertion knowledge and verification mechanisms, achieving a 147% increase in correct test cases over AthenaTest. Shin et al. [7] fine-tuned CodeT5 with project-level domain adaptation, improving line coverage (18.62%) over search-based tools like EvoSuite [8]. PyTester [9] introduced a reinforcement learning-based approach for generating test cases from natural language, outperforming GPT-3.5 [10] and StarCoder [11] on the APPS benchmark. Meanwhile, Tufano et al. [12] demonstrated the potential of pretrained transformers for generating assert statements, achieving a 62% match rate with developer-written assertions. These advancements highlight the growing synergy between DL-based and search-based methods in automated test generation.

A. Our Contribution

While prior works like A3Test and PyTester have advanced automated test generation, most focus on generalized codebases and rely on either natural language or code inputs, rather than leveraging both. Our study differentiates itself in key ways:

1. Model Diversity: We compare three transformer-based models—CodeT5, CodeBERT, and CodeGen—highlighting how architecture, dataset size, and pretraining objectives impact test generation.

2. Targeted Dataset: A curated dataset of Python functions with natural language descriptions and unit tests evaluates models across diverse tasks.

3. Comprehensive Metrics: We assess both NLP-based (BLEU, ROUGE-L, Levenshtein) and software-testing metrics (code coverage) to measure test case quality.

4. Performance Insights: Contrary to expectations, CodeT5 outperforms CodeBERT and CodeGen. We show that CodeT5 offers better semantic alignment and more executable test cases. Statistical tests highlight that pretraining objectives and architecture are more crucial than model size or specialization in data-constrained settings.

III. Methodology

This section presents our automated experimental unit test generation framework using CodeT5, CodeBERT, and Code-

Gen. Each model processes a Python function concatenated with its docstring, capturing both structure and intended behavior, and outputs a unit test to validate correctness. Figure 1 provides an overview of the test generation pipeline, while Table I summarizes dataset, model, and hyperparameter details.

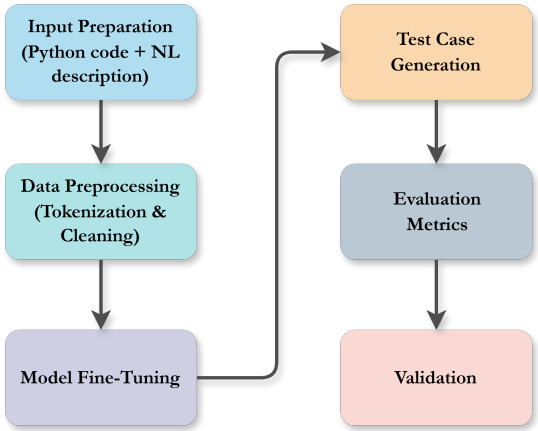

Fig. 1. Pipeline for automated unit test case generation. The process involves input preparation, data preprocessing, model fine-tuning, test case generation, evaluation metrics computation, and validation.

A. Dataset and Preprocessing

Our dataset comprised 3,123 manually generated samples, each containing a Python function, its docstring, and a corresponding unit test. This dataset evaluates model adaptability under resource constraints and is designed to reflect real-world, data-scarce scenarios. We split the data 60:20:20 into 1,873 training, 626 validation, and 626 test samples. Preprocessing included loading, cleaning, and tokenizing the data to match model input requirements. CodeT5 used AutoTokenizer with T5ForConditionalGeneration, CodeBERT utilized AutoTokenizer with EncoderDecoderModel, and CodeGen was processed with AutoTokenizer and CodeGenForCausalLM.

B. Fine-Tuning & Validation Methodology

The learning rates, batch sizes, and epochs were selected based on established practices for transformer-based models. For CodeT5, we adopted a learning rate of 5×10^{-5} and batch size 8, per its original pre-training setup. The lower learning rate (3×10^{-5}) and the larger batch size (16) of CodeBERT were chosen to balance convergence speed and stability. The learning rate of CodeGen (2×10^{-5}) and the small batch size (4) mitigated the risks of overfitting in autoregressive generation tasks. Early stopping was applied when validation metrics plateaued. The fine-tuning process was performed on a computational setup consisting of 4 NVIDIA Tesla V100 GPUs, 16 CPU cores, and 32GB of VRAM. To evaluate the impact of hyperparameters, we performed ablation studies of learning rates ($\pm 50\%$) and batch sizes ($\pm 2\times$) for CodeT5. The results show CodeT5 robustness: BLEU and code coverage remained stable ($\pm 3\%$) across configurations. CodeGen exhibited higher sensitivity: reducing its learning rate below 1×10^{-5} caused assertion errors to rise by 25%, while larger batch sizes degraded coverage by 15%. CodeBERT performance fell dramatically ($\sim 20\%$ in the F1 score) with learning rates above 5×10^{-5}, in accordance with its pretraining restrictions.

C. Inference & Post-Processing

After fine-tuning, the models were tested on the held-out test set to generate unit test cases. The same metrics used during validation were computed for the generated test cases to measure their performance. To further validate the generated unit tests, the predicted cases were combined with their corresponding Python functions to create test suites. These test suites were executed using pytest [14], and code coverage was measured using coverage.py [15].

IV. Results

In this section, we analyze the results of our experiments to evaluate the performance of CodeT5, CodeBERT, and CodeGen in generating Python unit tests. Table II provides a summary of the key findings.

A. Training & Validation Loss

The training and validation losses for the three models, CodeBERT, CodeT5, and CodeGen, provide insights into their convergence behavior. CodeT5 achieved the lowest average training loss of 0.0463 and validation loss of 0.0309, indicating effective learning and generalization capabilities. CodeBERT followed with average training and validation losses of 0.1297 and 0.0783, respectively, showing moderate learning efficiency. In contrast, CodeGen exhibited higher training and validation losses at 0.3000 and 0.3282.

B. Levenshtein Distance & Cosine Similarity

Levenshtein distance quantifies the minimum character edits needed to transform one string into another, with a smaller value indicating greater similarity. It is defined as:

$$d_{\text{Lev}}(i,j) = \begin{cases} \max(i,j), & \text{if } \min(i,j) = 0, \\ \min\big(& d_{\text{Lev}}(i-1,j)+1, \\ & d_{\text{Lev}}(i,j-1)+1, \\ & d_{\text{Lev}}(i-1,j-1)+\delta(x_i,y_j)\big), \\ \text{otherwise.} & \end{cases} \tag{1}$$

Here, x_i and y_j are characters from the two strings, and $\delta(x_i, y_j)$ is 0 if the characters are the same and 1 otherwise. A smaller Levenshtein distance indicates greater similarity between strings. Cosine similarity measures the angular cosine between two vectors, indicating semantic similarity, with a score closer to 1 showing higher similarity:

$$\text{Cosine Similarity} = \frac{\vec{A} \cdot \vec{B}}{\|\vec{A}\|\|\vec{B}\|} \tag{2}$$

TABLE I
SUMMARY OF DATASET, MODELS, AND TRAINING DETAILS

Aspect	Details
Dataset Size	3,123 samples
Train, Validation, Test Split	1,873 (train), 626 (validation), 626 (test)
Preprocessing	Tokenization and cleaning using model-specific tools
Models Used	CodeT5 (Salesforce/codet5-base), CodeBERT (huggingface/CodeBERTa-small-v1), CodeGen (Salesforce/codegen-350M-mono)
Fine-Tuning Parameters	CodeT5: 5×10^{-5}, Batch size: 8, Epochs: 6; CodeBERT: 3×10^{-5}, Batch size: 16, Epochs: 8; CodeGen: 2×10^{-5}, Batch size: 4, Epochs: 10
Infrastructure	4 NVIDIA Tesla V100 GPUs, 16 CPU cores, 32GB VRAM
Convergence Criteria	Monitored validation loss and evaluation metrics (BLEU, ROUGE-L, code coverage)
Evaluation Metrics	Levenshtein, Cosine similarity, BLEU, ROUGE-L, F1 Score, Code Coverage
Post-Processing	Test suite execution using pytest, coverage measured with coverage.py

where $\vec{A}$ and $\vec{B}$ are vector representations of two strings. A cosine similarity score closer to 1 indicates greater semantic similarity.

In our study, Levenshtein distance evaluates structural accuracy, while cosine similarity assesses semantic closeness. CodeT5 outperformed others with a Levenshtein score of 0.8535 and cosine similarity of 0.9008, indicating both structural and semantic alignment with the ground truth. CodeBERT showed moderate results (Levenshtein: 0.5561, Cosine: 0.5806), while CodeGen lagged behind (Levenshtein: 0.4329, Cosine: 0.3772), requiring more human intervention and data augmentation. Figure 2 shows the results.

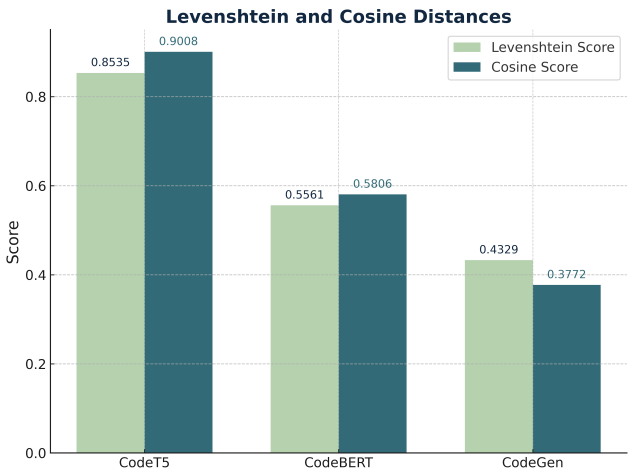

Fig. 2. Levenshtein and cosine similarity scores for the generated test cases. CodeT5 achieves the highest syntactic and semantic similarity to the ground truth, followed by CodeBERT and CodeGen.

C. Precision, Recall, & F1-Score

Precision is the ratio of correctly predicted positive cases to total predicted positives, reflecting the model's ability to avoid false positives. Recall measures the proportion of correctly predicted positives to actual positives, showing the model's ability to avoid false negatives. The F1-score is the harmonic mean of precision and recall, balancing both metrics. These metrics evaluate the relevance and correctness of generated unit test cases, ensuring they are valid (precision), comprehensive (recall), and of high quality (F1-score). From our experimental results, CodeT5 achieved the highest precision (0.8131), recall (0.8573), and F1-score (0.8337), reflecting its ability to generate accurate and complete unit test cases with our limited dataset. CodeBERT exhibited moderate performance with precision (0.6019), recall (0.7441), and F1-score (0.6646), indicating its limitations in ensuring both correctness and coverage. CodeGen, with much lower precision (0.2500), recall (0.4500), and F1-score (0.3214), struggled in our experimental setup. Figure 3 gives the summary of the results.

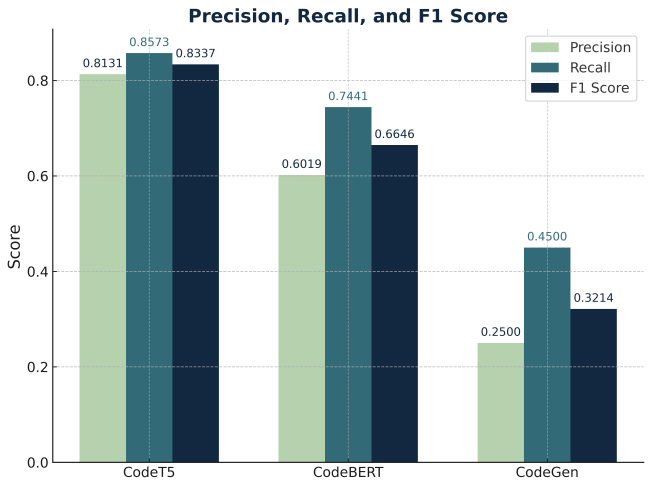

Fig. 3. Precision, recall, and F1-score across CodeT5, CodeBERT and CodeGen.

D. ROUGE-L & BLEU Scores

ROUGE-L and BLEU are key metrics for evaluating the quality of text generation models, focusing on the syntactic and semantic alignment between generated and reference sequences.

ROUGE-L measures the longest common subsequence (LCS) between generated and reference texts, reflecting structural similarity. The ROUGE-L score is defined as:

$$\text{ROUGE-L} = \frac{(1+\beta^2) \cdot \text{Precision}_{\text{LCS}} \cdot \text{Recall}_{\text{LCS}}}{\beta^2 \cdot \text{Precision}_{\text{LCS}} + \text{Recall}_{\text{LCS}}},$$

where $\text{Precision}_{\text{LCS}}$ and $\text{Recall}_{\text{LCS}}$ are based on the LCS, and β adjusts recall's weight.

BLEU evaluates n-gram overlap, emphasizing word-level similarity. It is computed as:

$$\text{BLEU} = \text{BP} \cdot \exp\left(\sum_{n=1}^{N} w_n \cdot \log p_n\right),$$

where p_n is n-gram precision, w_n are weights, and BP penalizes shorter outputs.

In our study, ROUGE-L assesses structural similarity, while BLEU measures fluency and correctness. CodeT5 achieved the highest scores (ROUGE-L: 0.8249, BLEU: 0.6646), indicating strong structural and semantic alignment. CodeBERT showed moderate scores (ROUGE-L: 0.5301, BLEU: 0.3445), suggesting partial alignment and fluency. CodeGen performed the worst (ROUGE-L: 0.2019, BLEU: 0.3507), highlighting its limitations and the need for more human intervention.

E. Code Coverage

Code coverage is a critical metric in software testing, measuring the extent to which the codebase is exercised by the executed test cases. Higher code coverage indicates that the generated test cases effectively validate the underlying code, minimizing potential bugs and vulnerabilities. In our study, code coverage was computed using the coverage.py tool, which provides detailed execution analysis for Python programs. In our experiments, CodeT5 achieved the highest code coverage at 75%. CodeBERT achieved a moderate code coverage of 48% and CodeGen scored only 10%, which was way below are initial assumption. But we attribute that result to the limited dataset used in the experiment. Figure 4 provides an overview of the BLEU, ROUGE-L, and code coverage scores.

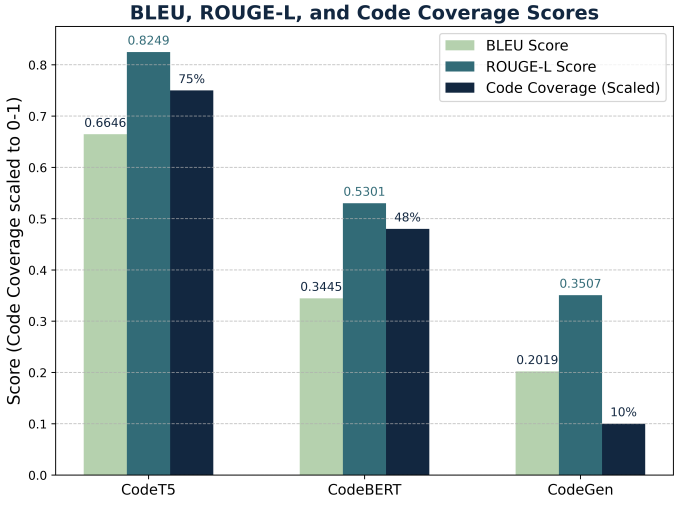

Fig. 4. ROUGE-L and BLEU scores for the generated test cases. CodeT5 exhibits the highest scores, indicating strong structural alignment in generated outputs and CodeT5 achieves the highest coverage at 75%.

F. Efficiency

To address efficiency, we also report the total time required to generate all test cases for each model in Table II. CodeT5 takes 10.29 minutes, CodeBERT takes 9.38 minutes, and CodeGen takes 7.25 minutes. More industrial analysis in real-world deployment might be needed to further validate the results.

G. Significance Testing with Independent T-tests

T-tests are statistical methods used to determine whether the means of two groups are significantly different from each other. Independent t-tests were conducted to evaluate the statistical significance of performance differences between the models (CodeT5, CodeBERT, and CodeGen) across key metrics such as F1-score, BLEU, ROUGE-L, and code coverage. The independent t-test compares the differences in metric scores for the same samples generated by two models, considering the mean difference ($\bar{d}$) and the standard deviation (s_d) of the differences. The t-statistic is calculated as:

$$t = \frac{\bar{d}}{s_d/\sqrt{n}}$$

where n is the number of samples. The resulting p-value indicates the probability of observing such differences under the null hypothesis, which assumes no difference between the models.

Table III presents the results of the independent t-tests. The p-values demonstrate that CodeT5 outperforms CodeBERT and CodeGen across all metrics (p-value < 0.05). The differences between CodeBERT and CodeGen are also statistically significant for most metrics, although less pronounced compared to CodeT5. This analysis ensures that the observed performance differences are not due to random variations.

V. Inference & Analysis of Results

In addition to the quantitative evaluations in Section IV, we qualitatively assessed the unit tests generated by CodeT5, CodeBERT, and CodeGen on unseen data. The ground-truth code (Figure 5) implements Floyd's Tortoise and Hare algorithm to detect duplicates, with the reference test checking for different duplicates in two arrays. CodeT5's output closely matches the ground-truth test, with minor input array adjustments (e.g., `[1,2,2,4]` and `[1,3,3,4]`), correctly identifying duplicates and maintaining syntactic and semantic fidelity. The assertions cover edge cases (e.g., duplicates at different positions) and mirror the input function's logic, reflecting CodeT5's ability to infer test logic from docstrings. CodeBERT generates a single test for a duplicate `4` in `[1,2,3,4,4]`, but lacks coverage of other scenarios. This partial coverage is reflected in the t-test results, where CodeBERT outperforms CodeGen but remains statistically inferior to CodeT5. CodeBERT's output shows an understanding of duplicates but lacks edge case exploration. Initially, CodeGen produced invalid outputs with no measurable coverage, but after fine-tuning, it generated a sparse yet functional test case, checking one duplicate and slightly increasing coverage. However, CodeGen's output remained less comprehensive, with higher p-values in t-tests compared to CodeT5 and CodeBERT. This reflects autoregressive models' tendency to prioritize

TABLE II
SUMMARY OF RESULTS ACROSS MODELS

Model	Validation Loss	Levenshtein	Cosine	BLEU	ROUGE-L	Precision	Recall	F1	Code Coverage (%)	Total Time to Generate Tests - GPU (Min)
CodeT5	0.0309	0.8535	0.9008	0.6646	0.8249	0.8131	0.8573	0.8337	75	10.29
CodeBERT	0.0783	0.5561	0.5806	0.3445	0.5301	0.6019	0.7441	0.6646	48	9.38
CodeGen	0.3282	0.4329	0.3772	0.2019	0.3507	0.2500	0.4500	0.3214	10	7.25

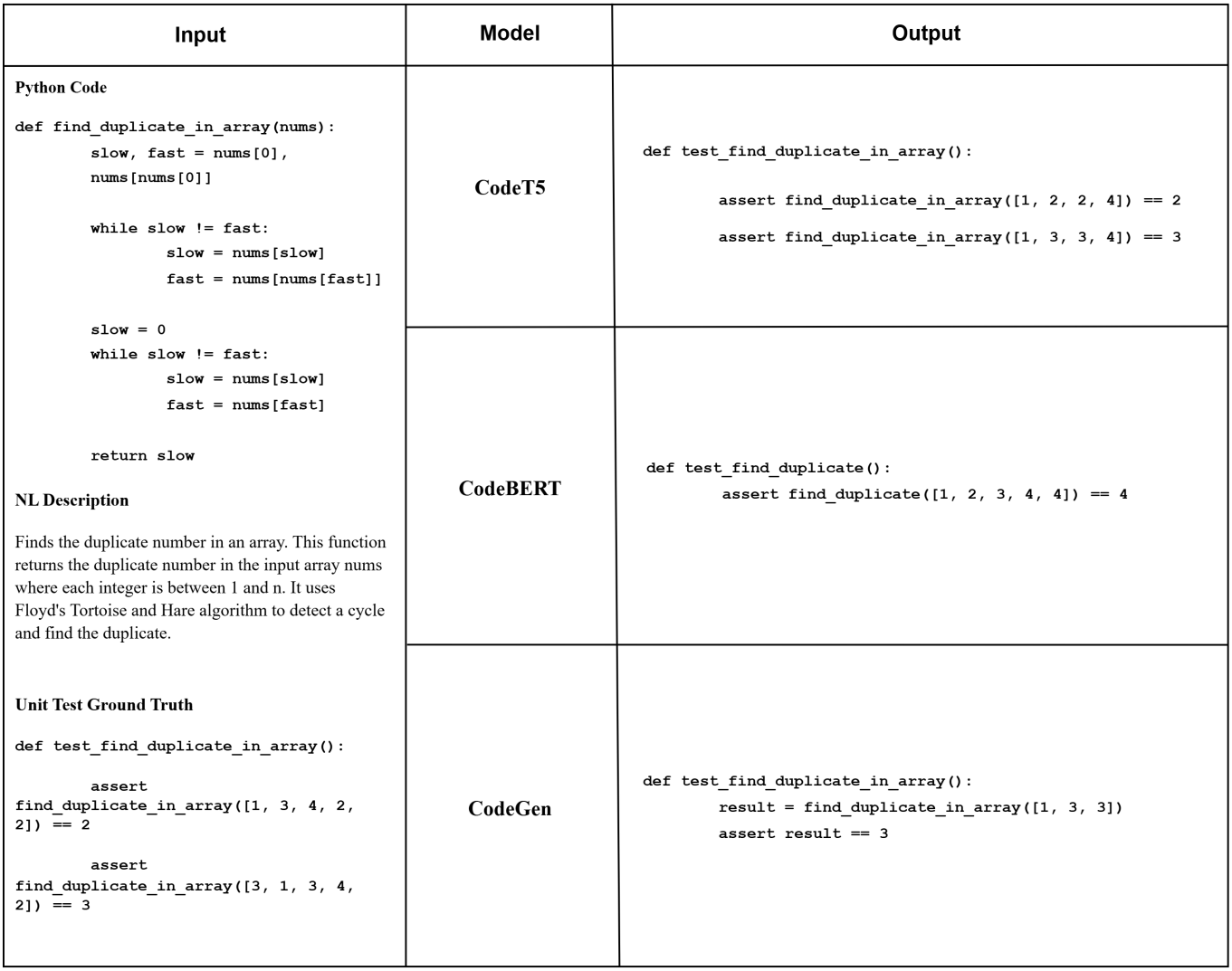

Fig. 5. Comparison of ground truth and model-generated unit test cases for a Python function implementing Floyd's Tortoise and Hare algorithm. The diagram highlights the varying levels of fidelity and coverage achieved by CodeT5, CodeBERT, and CodeGen, demonstrating CodeT5's alignment with the ground truth, CodeBERT's partial coverage, and CodeGen's output.

fluency over correctness in structured tasks. CodeGen's underperformance is due to its **autoregressive generation** approach, which struggles to enforce structured syntax (e.g., assertions and test logic) compared to CodeT5's encoder-decoder design. Its causal language modeling objective prioritizes fluency over structural fidelity, exacerbating errors in data-scarce settings. Manual inspection revealed that a large portion of CodeGen's tests contained incorrect assertions, while many had syntax errors, aligning with findings that autoregressive models sometimes falter in low-data regimes. Incorporating constrained decoding or larger datasets could mitigate these limitations. CodeT5's better performance stems from its encoder-decoder architecture, which enables bidirectional context for aligning input functionality with test logic. Unlike autoregressive models, CodeT5 jointly encodes inputs (code/docstrings) and decodes structured test cases. This is evident in its higher semantic similarity scores and code coverage. CodeT5's attention patterns focus on key elements like function signatures and docstring descriptions, leading to more accurate assertions.

TABLE III
PAIRWISE T-TEST RESULTS FOR MODEL PERFORMANCE METRICS

Metric	Comparison	Mean Diff.	p-value
F1-Score	CodeT5 vs CodeBERT	0.1691	0.003
	CodeT5 vs CodeGen	0.5123	0.0001
	CodeBERT vs CodeGen	0.3432	0.009
BLEU	CodeT5 vs CodeBERT	0.3201	0.002
	CodeT5 vs CodeGen	0.4646	0.0001
	CodeBERT vs CodeGen	0.1445	0.018
ROUGE-L	CodeT5 vs CodeBERT	0.2948	0.001
	CodeT5 vs CodeGen	0.4749	0.0001
	CodeBERT vs CodeGen	0.1801	0.020
Levenshtein	CodeT5 vs CodeBERT	0.2974	0.003
	CodeT5 vs CodeGen	0.4206	0.0001
	CodeBERT vs CodeGen	0.1232	0.025
Cosine Sim.	CodeT5 vs CodeBERT	0.3202	0.002
	CodeT5 vs CodeGen	0.5236	0.0001
	CodeBERT vs CodeGen	0.2034	0.017
Coverage	CodeT5 vs CodeBERT	27.0	0.005
	CodeT5 vs CodeGen	65.0	0.0001
	CodeBERT vs CodeGen	38.0	0.007

VI. LIMITATIONS & FUTURE WORK

By integrating semantic and execution-based metrics, this study provides a framework for evaluating AI-driven test case generation tools, offering insights for future NLP research in software development. However, limitations include the small dataset size and reliance on static metrics like BLEU and code coverage. Future work could incorporate runtime execution data (e.g., traces, edge case coverage) using tools like `pytest` and `coverage.py`, introduce defect detection metrics like mutation testing [16], expand datasets to include complex functions and multi-language support, and integrate human feedback for edge cases and complex logic.

VII. CONCLUSION

This study evaluated automated Python unit test generation using three transformer-based models—CodeT5, CodeBERT, and CodeGen—selected for their diverse architectures and design goals. While CodeGen (350M parameters) was expected to excel due to its broad code generation capabilities and CodeBERT (125M parameters) appeared well-suited for code understanding, CodeT5 delivered the most effective results, requiring minimal human intervention. These findings throw more light on the assumption that larger models or those pretrained specifically for code generation tasks will necessarily outperform smaller or more general-purpose ones. In practice, smaller models like CodeT5 can serve as resource-efficient alternatives to larger LLMs (e.g., LLaMA or GPT-3.5) while excelling in targeted tasks like test case generation. Although CodeGen struggled to produce functional tests consistently and CodeBERT achieved moderate success, both models can be valuable supplementary tools with human refinement, reducing effort and time in software development workflows. By integrating semantic metrics (e.g., BLEU, ROUGE-L) with execution-based measures like code coverage, this study provides a framework for evaluating AI-driven test generation tools. It serves as a reference for using transformers or LLMs in unit testing and lays the groundwork for future NLP research in software development and testing.

VIII. ACKNOWLEDGMENT

This work was funded by the BMBF Projects, KI4Boardnet and GENIAL. The authors acknowledge that AI writing tools were strictly used to improve the linguistic clarity of this manuscript, specifically to correct grammar, spelling, and sentence structure.

REFERENCES

[1] X. Li, A. Narechania, S. Kim, A. Kalyan, E. Reif, M. Chalaki, Y. Liu, W. J. Dally, B. Catanzaro, and C. Koksal, "CodeGen: An Open Large Language Model for Code," *Salesforce*, 2022. [Online]. Available: https://github.com/salesforce/CodeGen

[2] Z. Feng, D. Guo, D. Tang, N. Duan, X. Feng, M. Gong, L. Shou, B. Qin, T. Liu, and D. Jiang, "CodeBERT: A Pre-Trained Model for Programming and Natural Languages," in *Proceedings of the 2020 Conference on Empirical Methods in Natural Language Processing (EMNLP)*, 2020, pp. 1536–1547.

[3] Y. Wang, W. Shin, T. Jin, S. Wang, J. Ren, D. Tarlow, and C. Bui, "CodeT5: Identifier-aware Unified Pre-trained Encoder-Decoder Models for Code Understanding and Generation," in *Proceedings of the 2021 Conference on Empirical Methods in Natural Language Processing (EMNLP)*, 2021, pp. 8696–8708.

[4] S. Lukasczyk, G. Fraser, "Pynguin: Automated Unit Test Generation for Python," arXiv preprint, 2022.

[5] M. Tufano, D. Drain, A. Svyatkovskiy, S. K. Deng, N. Sundaresan, "AthenaTest: Transformer-Based Unit Test Case Generation," Microsoft Research, 2021.

[6] S. Alagarsamy, C. Tantithamthavorn, and A. Aleti, "A3Test: Assertion-Driven Test Case Generation with Domain Adaptation," Monash University, Australia, 2023.

[7] J. Shin, S. Hashtroudi, H. Hemmati, and S. Wang, "Domain Adaptation with Transformer Models for Test Case Generation," York University, Toronto, Canada, 2023.

[8] Gordon Fraser and Andrea Arcuri, "EvoSuite: Automatic Test Suite Generation for Object-Oriented Software," *Proceedings of the 19th ACM SIGSOFT Symposium and the 13th European Conference on Foundations of Software Engineering (ESEC/FSE)*, ACM, 2011, pp. 416–419, Szeged, Hungary, doi: 10.1145/2025113.2025165.

[9] W. Takerngsaksiri, R. Charakorn, C. Tantithamthavorn, and Y.-F. Li, "PyTester: A Reinforcement Learning-Based Text-to-Testcase Generation Framework," Monash University, Australia, 2023.

[10] OpenAI, "GPT-3.5: ChatGPT (March 2023 version)," *https://openai.com*, accessed January 2025.

[11] BigCode Project, "StarCoder: Open-Access Large Language Models for Code," *arXiv preprint arXiv:2305.06161*, 2023. Available at: https://arxiv.org/abs/2305.06161.

[12] M. Tufano et al., "Generating Accurate Assert Statements for Unit Test Cases Using Pretrained Transformers," Proceedings of the 3rd ACM/IEEE International Conference on Automation of Software Test, 2022.

[13] Adam Paszke, Sam Gross, Francisco Massa, Adam Lerer, James Bradbury, Gregory Chanan, Trevor Killeen, Zeming Lin, Natalia Gimelshein, Luca Antiga, Alban Desmaison, Andreas Kopf, Edward Yang, Zach DeVito, Martin Raison, Alykhan Tejani, Sasank Chilamkurthy, Benoit Steiner, Lu Fang, Junjie Bai, and Soumith Chintala, "PyTorch: An Imperative Style, High-Performance Deep Learning Library," *Advances in Neural Information Processing Systems (NeurIPS)*, 2019, pp. 8024–8035. Available at: https://pytorch.org.

[14] Holger Krekel, "pytest: Simple and Scalable Testing with Python," *https://pytest.org*, accessed January 2025.

[15] Ned Batchelder, "coverage.py: Code Coverage Measurement for Python," *https://coverage.readthedocs.io*, accessed January 2025.

[16] Y. Jia and M. Harman, "An Analysis and Survey of the Development of Mutation Testing," *IEEE Transactions on Software Engineering*, vol. 37, no. 5, pp. 649–678, 2011, doi: 10.1109/TSE.2010.62.